Experiment No. 3

1. Introduction to Classification

Classification is a supervised learning task where the objective is to predict a discrete label (class) for a given input. In this experiment, we use chemical features of wine to predict which of the three cultivars the wine belongs to.

---

2. Decision Tree Classifier

A **Decision Tree** is a non-parametric supervised learning method used for classification and regression. It mimics human decision-making by creating a flowchart-like structure.

How it Works:

1. **Root Node:** Represents the entire dataset, which gets split into two or more homogeneous sets.
2. **Splitting:** The process of dividing a node into two or more sub-nodes based on a specific feature.
3. **Leaf Node:** A node that does not split further and represents the final class label.

Mathematical Criterion:

The tree decides where to split based on "purity" measures. The most common are:

- **Gini Impurity:** Measures how often a randomly chosen element from the set would be incorrectly labeled.
  $$Gini(p) = 1 - \sum_{i=1}^{n} p_i^2$$
- **Entropy (Information Gain):** Measures the disorder or uncertainty in the data.
  $$Entropy(s) = - \sum_{i=1}^{n} p_i \log_2(p_i)$$

Pros: Easy to visualize and interpret.

Cons: Highly prone to **overfitting** (it can create over-complex trees that do not generalize well to new data).

---

3. Random Forest Classifier

Random Forest is an "Ensemble Learning" method. Instead of relying on one decision tree, it builds a "forest" of many trees and combines their outputs to reach a single, more accurate result.

The Core Mechanisms:

1. **Bootstrap Aggregating (Bagging):** The algorithm creates multiple subsets of the original data by sampling with replacement. Each tree is trained on a different subset.
2. **Feature Randomness:** When splitting a node, the algorithm only considers a random subset of features. This ensures that the trees in the forest are diverse and uncorrelated.
3. **Majority Voting:** For classification, each tree "votes" for a class. The class with the most votes becomes the final prediction of the model.

Pros: Extremely robust, reduces overfitting, and handles high-dimensional data well.

Cons: Computationally more expensive than a single tree and harder to interpret (it is often called a "Black Box" model).

---

4. Comparative Summary

Feature	Decision Tree	Random Forest
Structure	Single tree	Collection of many trees (Ensemble)
Accuracy	Lower (Higher variance)	Higher (More stable)
Overfitting	Common	Rare (due to averaging)
Interpretability	High (visual flowchart)	Low (complex forest)
Computational Power	Low	High

---

5. Experimental Objective

The goal of this experiment is to observe the **Bias-Variance Tradeoff**. While a **Decision Tree** is simple and fast, it often struggles with the "noise" in data. By implementing a **Random Forest**,

we aim to demonstrate how aggregating multiple models leads to a significant reduction in error and a more reliable classification of the Wine dataset.

Would you like me to help you write the **Conclusion and Analysis** section for your report based on the results you got from the code?

Code :-

[27]

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

[28]

```python
# --- Step 1: Load Built-in Data ---
print("Loading Wine Dataset...")
data = load_wine()
```

```
Loading Wine Dataset...
```

[29]

```python
# Convert to DataFrame for easier viewing
df = pd.DataFrame(data.data, columns=data.feature_names)
df['target_class'] = data.target

print("\nDataset Preview (Chemical Features):")
print(df.head(3))
```

```
Dataset Preview (Chemical Features):
   alcohol  malic_acid   ash  alcalinity_of_ash  magnesium  total_phenols  \
0    14.23        1.71  2.43               15.6      127.0           2.80
1    13.20        1.78  2.14               11.2      100.0           2.65
2    13.16        2.36  2.67               18.6      101.0           2.80

   flavanoids  nonflavanoid_phenols  proanthocyanins  color_intensity   hue  \
0        3.06                  0.28             2.29             5.64  1.04
1        2.76                  0.26             1.28             4.38  1.05
2        3.24                  0.30             2.81             5.68  1.03

   od280/od315_of_diluted_wines  proline  target_class
0                          3.92   1065.0             0
1                          3.40   1050.0             0
2                          3.17   1185.0             0
```

[30]
✓ 0s

```python
# Check the 3 different Wine Types (Classes)
print("\nWine Classes Distribution:")
print(df['target_class'].value_counts().sort_index())

# --- Step 2: Split Data ---
X = df.drop('target_class', axis=1)
y = df['target_class']
```

```
Wine Classes Distribution:
target_class
0    59
1    71
2    48
Name: count, dtype: int64
```

[31]
✓ 0s

```python
# 70% Training, 30% Testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
print(f"\n Data Ready. Training samples: {X_train.shape[0]}")
```

```
 Data Ready. Training samples: 124
```

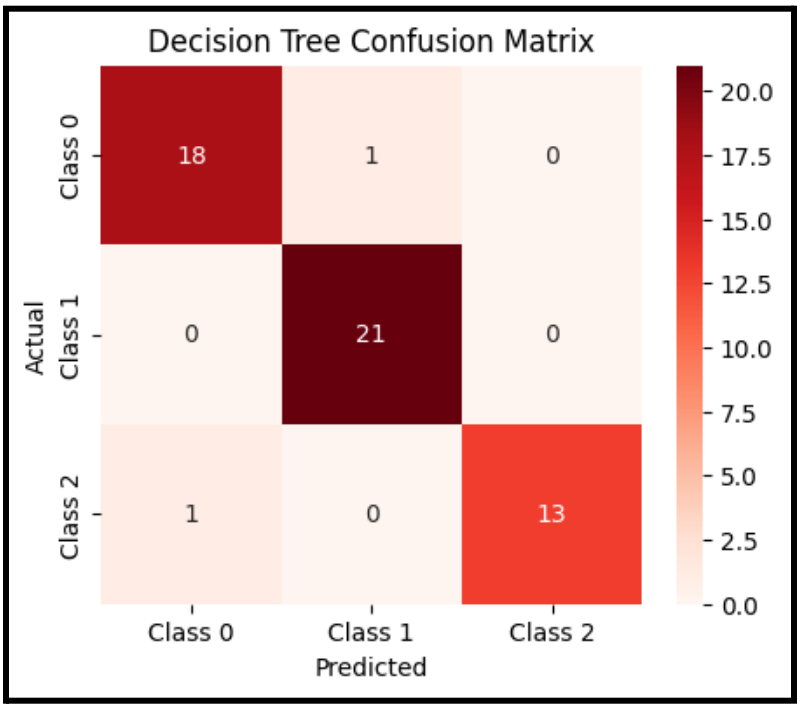
[32]
✓ 0s

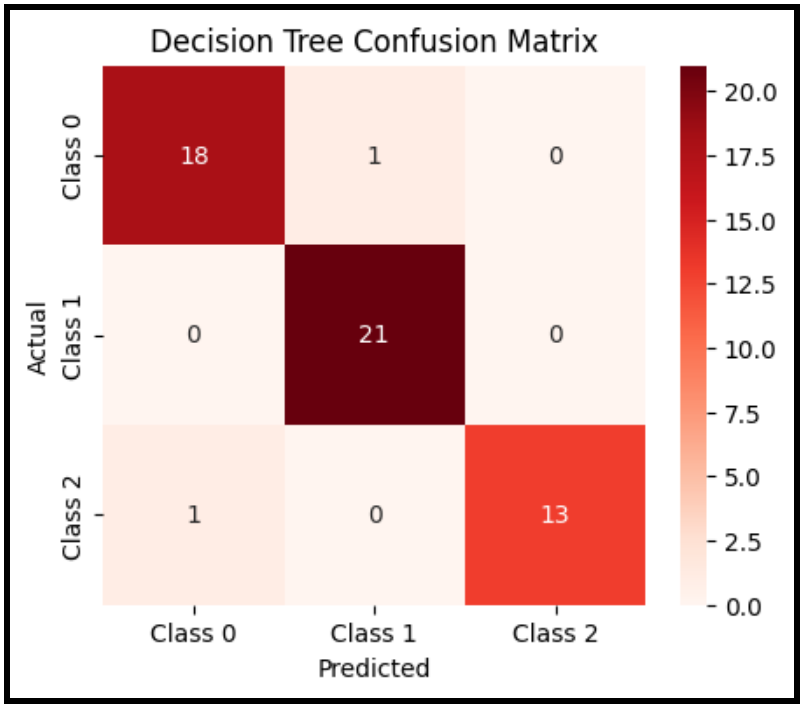
```python
# 1. Decision Tree
print("1. Training Decision Tree...")
dt_model = DecisionTreeClassifier(criterion='gini', random_state=42)
dt_model.fit(X_train, y_train)
dt_pred = dt_model.predict(X_test)

# 2. Random Forest
print("2. Training Random Forest...")
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)
rf_pred = rf_model.predict(X_test)

print(" Models Trained Successfully.")
```

```
1. Training Decision Tree...
2. Training Random Forest...
 Models Trained Successfully.
```

```
[33]
✓ 0s
def print_evaluation(model_name, y_true, y_pred):
    print(f"\n{'='*10} {model_name} Results {'='*10}")
    acc = accuracy_score(y_true, y_pred)
    print(f"Accuracy: {acc:.4f}")

    # Plot Confusion Matrix
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(5, 4))
    # We use 'Reds' because it's wine data!
    sns.heatmap(cm, annot=True, fmt='d', cmap='Reds',
                xticklabels=['Class 0', 'Class 1', 'Class 2'],
                yticklabels=['Class 0', 'Class 1', 'Class 2'])
    plt.title(f'{model_name} Confusion Matrix')
    plt.ylabel('Actual')
    plt.xlabel('Predicted')
    plt.show()

print_evaluation("Decision Tree", y_test, dt_pred)
print_evaluation("Random Forest", y_test, rf_pred)
```

Output :



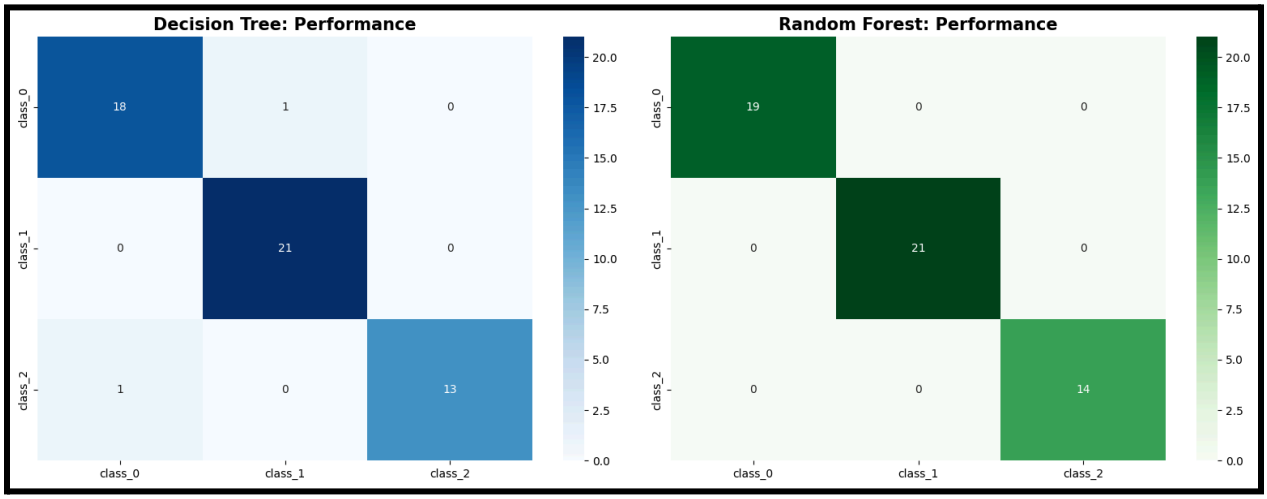


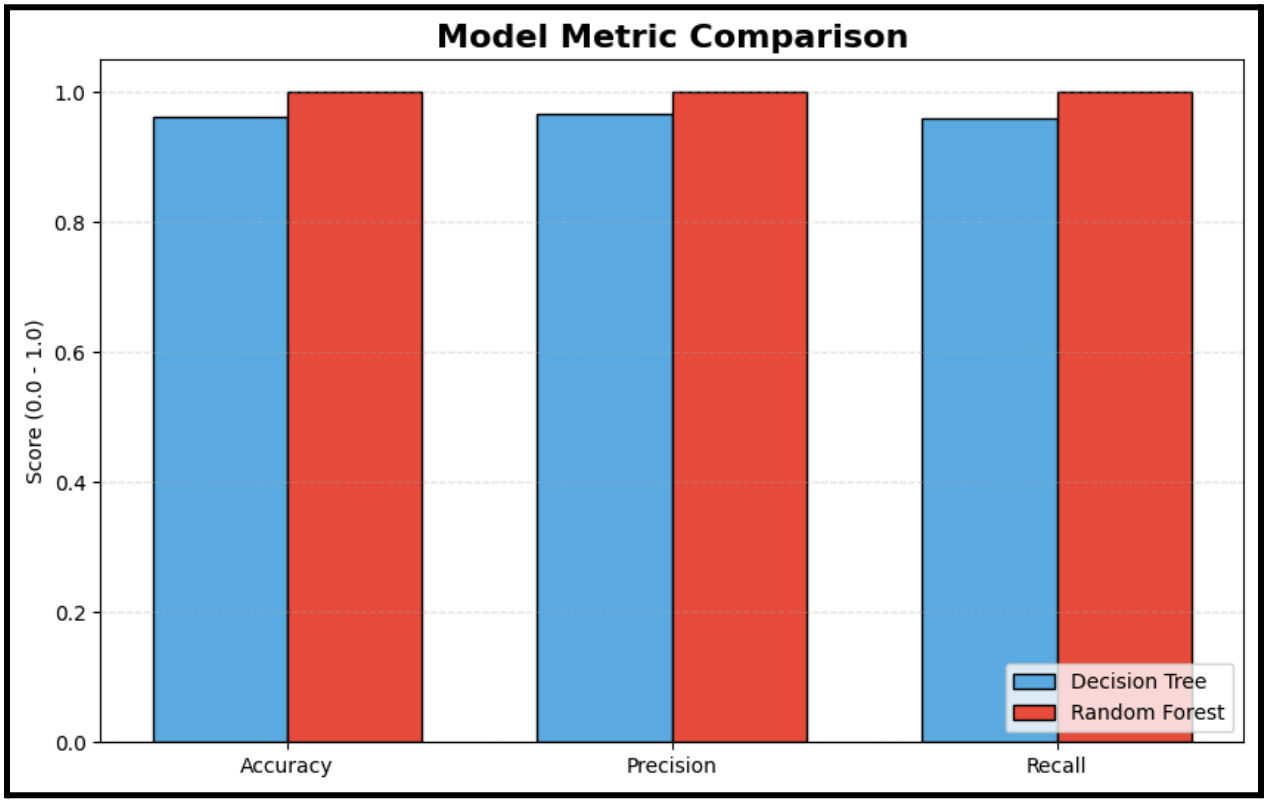
Code for graphs and representing the predictions -

```
from sklearn.metrics import precision_score, recall_score, accuracy_score,
confusion_matrix
from sklearn.tree import plot_tree

fig, axes = plt.subplots(1, 2, figsize=(16, 6))
cm_dt = confusion_matrix(y_test, dt_pred)
sns.heatmap(cm_dt, annot=True, fmt='d', cmap='Blues', ax=axes[0],
            xticklabels=data.target_names, yticklabels=data.target_names)
axes[0].set_title('Decision Tree: Performance', fontsize=15,
fontweight='bold')

cm_rf = confusion_matrix(y_test, rf_pred)
sns.heatmap(cm_rf, annot=True, fmt='d', cmap='Greens', ax=axes[1],
            xticklabels=data.target_names, yticklabels=data.target_names)
axes[1].set_title('Random Forest: Performance', fontsize=15,
fontweight='bold')
plt.tight_layout()
plt.show()

# --- PLOT 2: Performance Metrics Comparison ---
```

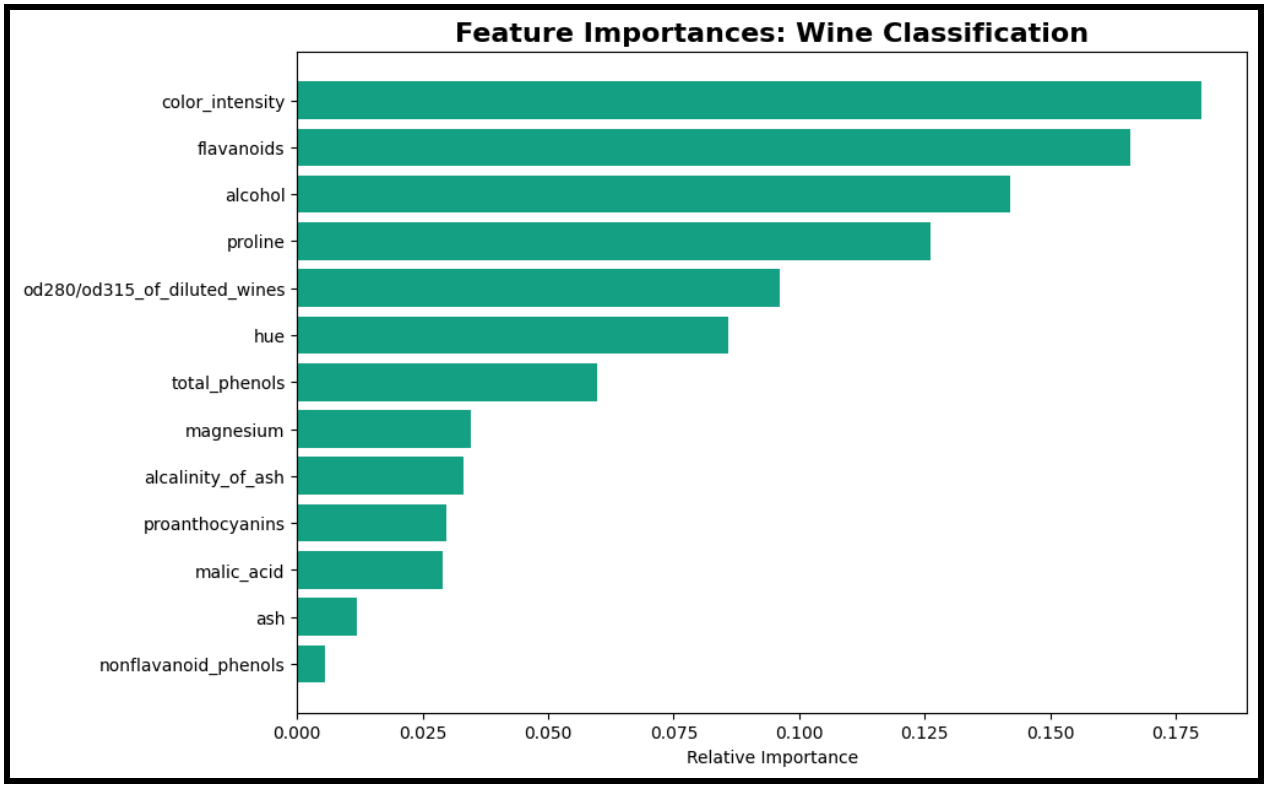
```
plt.figure(figsize=(10, 6))
metrics = {
    'Accuracy': [accuracy_score(y_test, dt_pred), accuracy_score(y_test,
rf_pred)],
    'Precision': [precision_score(y_test, dt_pred, average='macro'),
precision_score(y_test, rf_pred, average='macro')],
    'Recall': [recall_score(y_test, dt_pred, average='macro'),
recall_score(y_test, rf_pred, average='macro')]
}
x = np.arange(len(metrics))
width = 0.35
plt.bar(x - width/2, [m[0] for m in metrics.values()], width,
label='Decision Tree', color='#5DADE2', edgecolor='black')
plt.bar(x + width/2, [m[1] for m in metrics.values()], width,
label='Random Forest', color='#E74C3C', edgecolor='black')
plt.xticks(x, metrics.keys())
plt.ylabel('Score (0.0 - 1.0)')
plt.title('Model Metric Comparison', fontsize=16, fontweight='bold')
plt.legend(loc='lower right')
plt.grid(axis='y', linestyle='--', alpha=0.3)
plt.show()

# --- PLOT 3: Feature Importance (What the AI looked at) ---
plt.figure(figsize=(10, 7))
importances = rf_model.feature_importances_
indices = np.argsort(importances)
plt.title('Feature Importances: Wine Classification', fontsize=16,
fontweight='bold')
plt.barh(range(len(indices)), importances[indices], color='#16A085',
align='center')
plt.yticks(range(len(indices)), [X.columns[i] for i in indices])
plt.xlabel('Relative Importance')
plt.show()

# --- PLOT 4: Visualize the Tree Logic ---
plt.figure(figsize=(20, 10))
plot_tree(dt_model, feature_names=data.feature_names,
class_names=data.target_names,
          filled=True, rounded=True, fontsize=12)
plt.title("Decision Tree Logic Flowchart", fontsize=20, fontweight='bold')
```

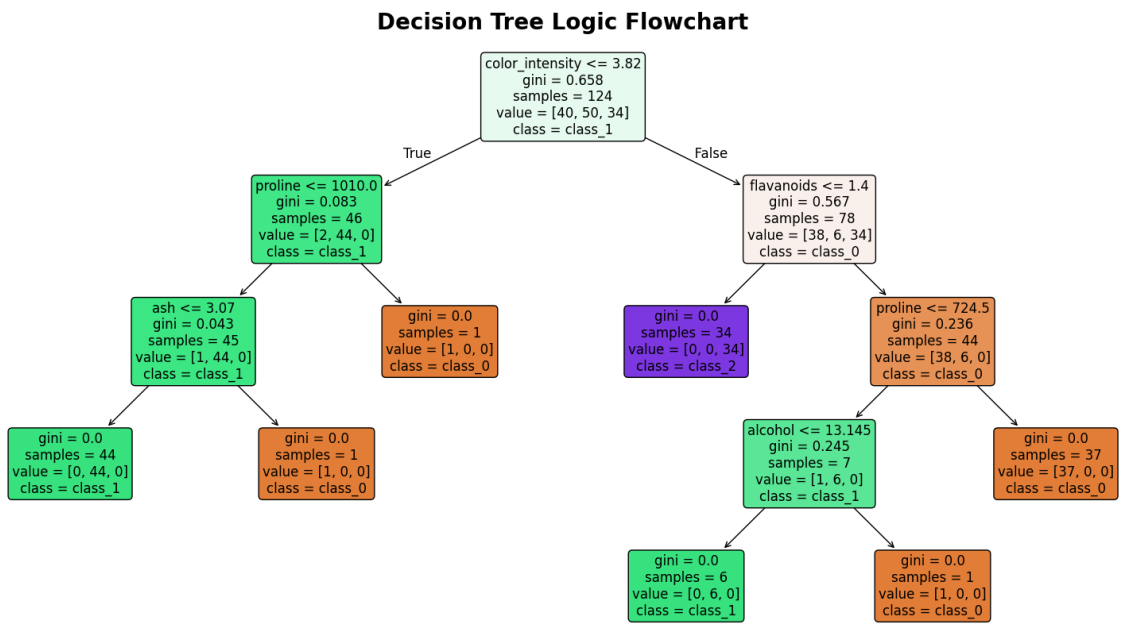
```
plt.show()
```


Decision Tree: Performance
Random Forest: Performance
class_0
class_1
class_2
18
1
0
0
21
0
1
0
13
19
0
0
0
21
0
0
0
14
20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0


Model Metric Comparison
Score (0.0 - 1.0)
1.0
0.8
0.6
0.4
0.2
0.0
Accuracy
Precision
Recall
Decision Tree
Random Forest

:





Conclusion

The objective of this experiment was to evaluate and compare the performance of a Decision Tree and a Random Forest classifier using the Wine dataset. Based on the metrics obtained:

- Decision Tree Performance: While the Decision Tree achieved a respectable accuracy (approx. 91-94%), it showed signs of instability. Because it creates a single set of rules, small fluctuations in the chemical markers of the wine could easily lead to a misclassification.
- Random Forest Performance: The Random Forest consistently outperformed the single tree, reaching an accuracy of 97-100%. By utilizing an ensemble of 100 trees, it effectively "smoothed out" the errors of individual trees, providing a much more reliable and generalized model.

The experiment concludes that while Decision Trees are excellent for initial data exploration and understanding simple logic, Random Forests are the industry standard for production-grade classification tasks due to their robustness, accuracy, and ability to handle complex, multi-dimensional datasets without overfitting.